

Reviewer Pack — Real Radical Dimension Bounds SOS
Refutation Degree for max-...
Entry 24f0050c445f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler
2026-05-20 19:52:46 UTC

Real Radical Dimension Bounds SOS Refutation Degree for
max-CUT

Entry ID: 24f0050c445f Verdict: INCONCLUSIVE Recorded: 2026-05-20 19:52:46 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry Field B (complexity object): SOS Refu-
tation Degree for max-CUT

Statement:

For a max-CUT instance with real radical variety of dimension d , the SOS refutation de-
gree is at least $\Omega(\log d)$.

Rationale (proposer’s reasoning):

The real radical’s dimension captures the complexity of real solutions, which may require
higher SOS degrees to refute. This bridge could expose geometric constraints on opti-
mization landscapes.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f4b23241d53b0841

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empir-
ical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        A = [random.randint(1, n-1) for _ in range(n)]
        B = [i for i in range(n) if i not in A]
        return A, B

    def matrix_multiplication(A, B):
        m, k = len(A), len(B[0])
        n = len(B)
        C = [[0] * k for _ in range(m)]
        for i in range(m):
            for j in range(k):
                for l in range(n):
                    C[i][j] += A[i][l] * B[l][j]
        return C

    def gaussian_elimination(A, b):
        n = len(b)
        augmented_matrix = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = max(range(i, n), key=lambda k: abs(augmented_matrix[k][i]))
            augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
            pivot = augmented_matrix[i][i]
            if pivot == 0:
                continue
            for j in range(n):
                augmented_matrix[i][j] /= pivot
            b[i] /= pivot
            for k in range(n):
                if k != i:
                    factor = augmented_matrix[k][i]
                    for j in range(n):
                        augmented_matrix[k][j] -= factor * augmented_matrix[i][j]
                    b[k] -= factor * b[i]
        return [row[:-1] for row in augmented_matrix], b

    def symbolic_rank_check(A):
        n = len(A)
        rank = 0
```

```

    for i in range(n):
        if all(abs(a) < 1e-9 for a in A[i]):
            continue
        rank += 1
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return rank

def sdp_relaxation(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                C[i][j] = A[i][i] + B[j][j]
            else:
                C[i][j] = -A[i][j] - B[j][i]
    _, b = gaussian_elimination(C, [1] * n)
    return max(b)

def log_base_2(x):
    if x <= 0:
        return float('-inf')
    return math.log2(x)

n = 40
A, B = generate_max_cut_instance(n)
d = symbolic_rank_check(A)
sos_refutation_degree = sdp_relaxation(A, B)

metric_name = "sos_refutation_degree"
metric_value = sos_refutation_degree
instances_tested = 1
conjecture_holds = sos_refutation_degree >= log_base_2(d + 1)
counterexample = "" if conjecture_holds else f"Graph with n={n}, A={A}, B={B}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44e5f9fe.py", line 111, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44e5f9fe.py", line 88, in run_trial
    d = symbolic_rank_check(A)
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44e5f9fe.py", line 60, in symbolic_rank_check
    if all(abs(a) < 1e-9 for a in A[i]):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError, preventing data collection. Support fraction cannot be evaluated. | next: Fix the TypeError in symbolic_rank_check by ensuring A[i] is iterable (e.g., convert to list or handle integer inputs)

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	78956
2	preregistration	ollama_remote	qwen3:8b	0	0	31557

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	qwen3:8b	0	0	27365
4	novelty	ollama_remote	qwen3:8b	0	0	18361
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11625
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14142
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8596
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12458
9	judge	ollama_remote	qwen3:8b	0	0	17565

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 220624 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/24f0050c445f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/24f0050c445f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/24f0050c445f.tar.gz` (if generated)